

Modern Computer Architecture

NST-CS-MCA504 | Semester 5 • 2026 | Complete Visual Cheatsheet & Revision Compendium

TOTAL LECTURES
14 Comprehensive Units

TOTAL PAGES
15 Pages (Cover + Notes)

COURSE TRACK
Semester 5 Examination Prep

TABLE OF CONTENTS & INDEX

UNIT	LECTURE TOPIC & SUMMARY	PAGE
L-01	Course Intro & Boolean Algebra Foundations	Page 02
L-02	Logic Minimisation & Universal Gates	Page 03
L-03	Combinational Building Blocks: CISC vs RISC	Page 04
L-04	Number Systems, Two's Complement & Overflow	Page 05
L-05	Sequential Logic: Feedback, Latches & Memory	Page 06
L-06	Latches, Flip-Flops & Timing Foundations	Page 07
L-07	Metastability, Synchronizers & Static Timing Analysis	Page 08
L-08	Flip-Flop Types, Registers & Digital Counters	Page 09
L-09	Register Files & Finite State Machines (FSMs)	Page 10
L-10	Memory Technologies: SRAM vs. DRAM Architecture	Page 11
L-11	Von Neumann vs. Harvard Architecture & Bottleneck	Page 12
L-12	ISA Design Principles: RISC vs. CISC Comparison	Page 13
L-13	MIPS32 & RISC-V Instruction Formats (R, I, J)	Page 14
L-14	Control Flow Translation & Stack Calling Conventions	Page 15

1. Welcome to Computer Architecture

1.1 The Promise and the Goal

- Reveals what is happening beneath every line of code.
- Goal: Understand how a computer executes a program from transistors, gates, registers, datapaths, assembly language up to Python.

1.2 Why Architecture Matters

- Performance:** Caches + pipelines $\Rightarrow$ 100x faster code
- AI/ML Optimization:** GPUs, TPUs, tensor cores are pure hardware architecture.
- Systems Depth:** OS, compilers, databases rest entirely on hardware execution model.

Key Insight

Computer architecture separates software engineers who merely **use** computers from systems engineers who deeply **understand** how computation is physically carried out.

2. Architecture vs. Organization

2.1 Core Definitions

- Architecture:** Attributes visible to programmer. Defines interface $\rightarrow$ what instructions, registers, memory addressing.
- Organization:** Operational units + interconnections realizing architectural specs. Deals with pipelines, caches, control signals, clock speeds. (Hidden from programmer)

2.2 The Programmer Contract

Architecture
(What is promised)
Ex: "This CPU supports ADD instruction."

Organization
(How it is implemented)
Ex: "ADD executed in 1 clock cycle using carry-lookahead adder."

Analogy



Architecture
Driver's interface
(Steering, pedals, dashboard)



Organization
Engineering under the hood
(Engine, pistons, gears)

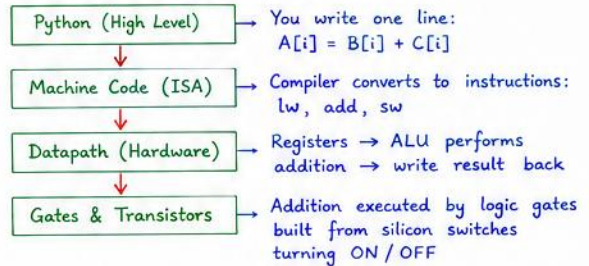
3. The Abstraction Stack

3.1 Layer-by-layer Breakdown

①	AI / ML Models	High Level (More Abstract)
②	Applications & Languages	
③	Operating System	
④	Instruction Set Architecture (ISA)	
⑤	Datapath & Control (Registers, ALU, FSM)	
⑥	Logic Gates (AND, OR, NOT, XOR)	Low Level (Less Abstract)
⑦	Transistors (Silicon Switches)	

3.2 Tracing a Single Line of Code

Example: $A[i] = B[i] + C[i]$



4. Boolean Basics & Logic Gates

4.1 Voltage Representation of 1 and 0

Logic	Voltage Level
1 (TRUE / HIGH)	Presence of voltage (~3.3V or 5V)
0 (FALSE / LOW)	Absence of voltage (~0V, Ground)

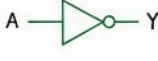
Computers are reliable because they only need to distinguish between two discrete voltage thresholds.

4.2 Fundamental Logic Gates (Truth Tables)

NOT Gate

Y = A'

A	Y
0	1
1	0



Output is inverse of input.

AND Gate

Y = A · B

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

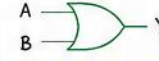


Output is 1 only when BOTH inputs are 1.

OR Gate

Y = A + B

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1



Output is 1 when AT LEAST ONE input is 1.

XOR Gate

Y = A ⊕ B

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

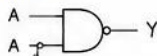


Output is 1 when inputs DIFFER.

4.3 NAND & NOR: Universal Gates

NAND (NOT-AND) and NOR (NOT-OR) are **UNIVERSAL GATES**.

- Using only NAND (or only NOR) we can build AND, OR, NOT gates $\rightarrow$ any logic circuit.
- NOT gate using NAND:



$$Y = A \text{ NAND } A = \text{NOT } A$$

Silicon Optimization

Manufacturers can optimize one gate type (NAND) to make it cheap, fast, dense $\rightarrow$ used to build all CPU operations.

5. Boolean Laws & Simplification

5.1 Core Algebraic Laws

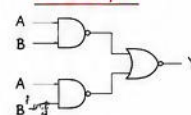
- Identity Laws** $A + 0 = A$ | $A \cdot 1 = A$
- Null / Dominance** $A + 1 = 1$ | $A \cdot 0 = 0$
- Idempotent Laws** $A + A = A$ | $A \cdot A = A$
- Complement Laws** $A + A' = 1$ | $A \cdot A' = 0$
- Absorption Law** $A + A \cdot B = A$
- De Morgan's Laws** $(A \cdot B)' = A' + B'$ | $(A + B)' = A' \cdot B'$

5.2 Worked Algebraic Minimization

Simplify: $Y = A \cdot B + A \cdot B'$

- Start: $Y = A \cdot B + A \cdot B'$
- Factor out A: $Y = A \cdot (B + B')$
- Complement Law: $B + B' = 1$
 $\rightarrow Y = A \cdot (1)$
- Identity Law: $A \cdot 1 = A$
 $\rightarrow Y = A$

Gates Before



(2 AND + 1 OR + 1 NOT)

After Simplification

$A \rightarrow Y$
(Just a wire!)

Silicon Saved

Reduced to a single wire. Saves gates, area, power, heat & increases speed.

1. WHY SHRINK A CIRCUIT? (SILICON EFFICIENCY)

1.1 Area, Power, Speed, and Cost

- **Area** : Fewer gates $\Rightarrow$ smaller die $\Rightarrow$ more chips / wafer $\Rightarrow$ lower fabrication cost.
- **Power** : Fewer switching $\Rightarrow$ less dynamic power $\Rightarrow$ longer battery life.
- **Speed** : Shorter gate chains $\Rightarrow$ less propagation delay $\Rightarrow$ higher clock frequency.
- **Cost** : Smaller area + higher yield $\Rightarrow$ more profit.

1.2 Industry Scale & Logic Synthesis

- Apple M3 chip $\rightarrow$ 98 billion transistors on TSMC 3nm. Even 1% gate reduction $\Rightarrow$ huge die space + \$ saving.
- **Tools** : Synopsys Design Compiler, Cadence Genus, etc.

SYNTHESIS POWER

Typical 4-variable function can shrink from
16 gates $\rightarrow$ as few as 2 gates
 (same truth table, 8x silicon saving)

2. TRUTH TABLE TO CANONICAL EXPRESSION (SOP & POS)

2.1 Sum of Products (SOP) & Minterms

- SOP ORs terms for rows where $F=1$.
- Each term = **minterm** (single row AND).
- **Rule** : $1 \rightarrow X$, $0 \rightarrow X'$
Keep if 1; invert if 0.
- **Shorthand** : $\Sigma m(\dots)$
Example : $F = \Sigma m(1, 3, 5, 7)$

WORKED SOP EXAMPLE (Row 5)

A	B	C	F
1	0	1	1

$A=1, B=0, C=1, F=1$
 $A=1 \rightarrow A$
 $B=0 \rightarrow B'$
 $C=1 \rightarrow C$
 $m_5 = A \cdot B' \cdot C$

2.2 Product of Sums (POS) & Maxterms

- POS ANDs terms for rows where $F=0$.
- Each term = **maxterm** (single row OR).
- **Rule** : $1 \rightarrow X'$, $0 \rightarrow X$
(Inverted relative to SOP).
- **Shorthand** : $\Pi m(\dots)$
Example : $F = \Pi m(0, 2, 4, 6)$

WORKED POS EXAMPLE (Row 2)

A	B	C	F
0	1	0	0

$A=0, B=1, C=0, F=0$
 $A=0 \rightarrow A$
 $B=1 \rightarrow B'$
 $C=0 \rightarrow C$
 $M_2 = (A + B' + C)$

3. KARNAUGH MAPS (K-MAPS) & MINIMISATION

3.1 Gray-Code Adjacency & Legal Group Sizes

- K-map arranges cells so adjacent cells differ by exactly one bit (Gray code).
- Column order (2 vars) : 00 01 11 10

Group Size	Cells	Variables Cancelled	Example Benefit
Single	1	0	No simplification
Pair	2	1	Remove 1 variable
Quad	4	2	Remove 2 variables
Octet	8	3	Remove 3 variables

3.2 The Torus Wrap-Around Trick & Don't-Cares

- K-maps behave like a **TORUS** (donut).
Left $\leftrightarrow$ right, Top $\leftrightarrow$ bottom, Corners are adjacent.

4-variable
K-map
(AB rows,
CD columns)

	00	01	11	10
00	m0	m1	m3	m2
01	m4	m5	m7	m6
11	m12	m13	m15	m14
10	m8	m9	m11	m10

Don't-Care (X) : Inputs that cannot occur physically (e.g. invalid BCD 10-15).
Treat X as 1 or 0 to form larger groups.

Wrap-Around

Columns 00 $\leftrightarrow$ 10
Rows 00 $\leftrightarrow$ 10
4 corners can form a Quad.

Ex: corners = 1
 $\Rightarrow F = B'D'$

Interpretation on K-map

Single cell $\rightarrow$ No cancel
 Pair (2) $\rightarrow$ 1 variable cancels
 Quad (4) $\rightarrow$ 2 variables cancel
 Octet (8) $\rightarrow$ 3 variables cancel
 (Larger group $\Rightarrow$ simpler term)

3.3 Algebra Laws mapped to K-Maps

- **Identity** : $A + 0 = A$, $A \cdot 1 = A$
- **Complement** : $A + A' = 1$, $A \cdot A' = 0$
- **Absorption** : $A + A \cdot B = A$, $A \cdot (A + B) = A$
- **Distributive** : $A(B + C) = AB + AC$
 $A + BC = (A + B)(A + C)$

4. DEMORGAN'S THEOREMS & BUBBLE PUSHING

4.1 Formal Theorems & Algebra

Theorem 1 : $(A \cdot B)' = A' + B'$
 (NOT of AND = OR of NOTs)

Theorem 2 : $(A + B)' = A' \cdot B'$
 (NOT of OR = AND of NOTs)

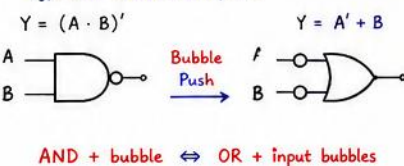
Memory Hook : "Break the bar, flip the sign."

Example : Simplify $(A \cdot B + C)'$

Step 1 : Break bar over OR
 $\rightarrow (A \cdot B)' \cdot C'$
Step 2 : Break bar over AND
 $\rightarrow (A' + B') \cdot C'$

4.2 Bubble Pushing (Visual)

- Pushing a bubble backward through a gate changes its type and bubbles on inputs.



5. UNIVERSAL GATES & FAB'S CMOS CHOICES

5.1 Implementing NOT, AND, OR using NAND / NOR

Using **NAND Gate** ($\downarrow$)

NOT (A)

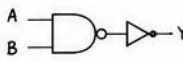
Tie inputs together



$Y = \text{NAND}(A, A)$
 $= A'$

AND (A, B)

Invert NAND output



$Y = \text{NOT}(\text{NAND}(A, B))$
 $= A \cdot B$

OR (A, B)

NAND inverted inputs



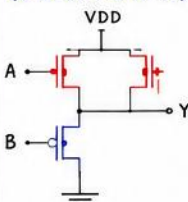
$Y = \text{NAND}(A', B')$
 $= (A' \cdot B')' = A + B$

Similarly, NOR gate is also universal.
 Use DeMorgan's theorem to derive.
 (NAND typically preferred in industry.)

5.2 CMOS Transistor Budget (NAND vs AND)

CMOS NAND

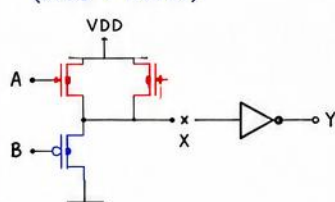
Needs only 4 transistors
 (2 PMOS + 2 NMOS)



Total = 4 Transistors

CMOS AND

Needs 6 transistors
 (NAND + Inverter)



Total = 6 Transistors

Scaling Transistors : With 28 billion transistors on Apple's M4, saving 2 transistors / AND gate $\Rightarrow$ massive silicon area saving, lower leakage, less heat, higher performance.

Combinational Building Blocks & ISA Philosophies

1 Why Reusable Building Blocks?

- Modern CPUs are not designed gate-by-gate.
- Engineers use a standardized catalogue of reusable building blocks (MUXes, decoders, adders, registers, etc.).
- Think in higher-level intent (e.g. "select input B") not in individual gates.

Key Idea:

Abstraction → Modularity → Reuse → Verification
→ Cascading Scale

4 Pillars of Block-Based Design:

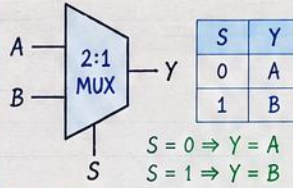
- 1 **Abstraction**: hides low-level gate complexity.
- 2 **Reuse**: one verified design used many times.
- 3 **Verification**: test once; trust all copies.
- 4 **Cascading Scale**: small blocks → larger blocks (e.g. 3x2:1 MUX → 4:1 MUX, 1-bit adders → 64-bit ALU).

A modern CPU with billions of transistors is built by repeating just ~5 core block types millions of times.

2 Multiplexers (MUX) & Universal Logic

2.1 2:1 MUX

$$Y = \bar{S} \cdot A + S \cdot B$$



- **Analogy**: Railway switch - S controls which input line (A or B) is routed to output.

2.3 MUX Can Implement Any Boolean Function

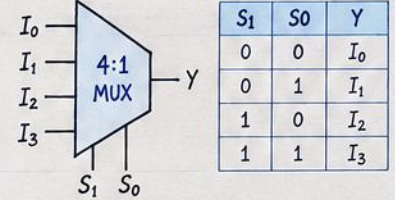
- A 2ⁿ:1 MUX can implement any n-variable Boolean function.
- Connect input variables to select lines.
- Set each data input ($I_0, I_1, \dots$) to 0 or 1 based on truth table.

Example: XOR ($A \oplus B$)

$S_1 = A, S_0 = B$
 $I_0 = 0, I_1 = 1, I_2 = 1, I_3 = 0$
 $\Rightarrow$ Output $Y = A \oplus B$

2.2 4:1 MUX

$$Y = \bar{S}_1 \bar{S}_0 I_0 + \bar{S}_1 S_0 I_1 + S_1 \bar{S}_0 I_2 + S_1 S_0 I_3$$



- Can be built using three 2:1 MUXes. This tree structure scales to 8:1, 16:1, 64:1 ...

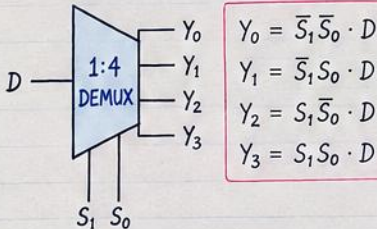
Real-World Use:

- This principle powers Lookup Tables (LUTs) in FPGAs, where programmable MUXes implement any custom circuit.

3 Demultiplexers & Decoders

3.1 1:4 DEMUX

Routes a single input to one of many outputs.



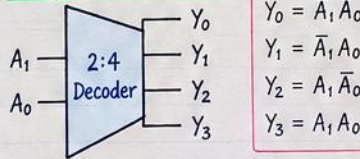
- All unselected outputs remain 0.

Use: CPU writing a result into one specific register in a register file (based on address).

3.2 n-to-2ⁿ Decoder

- Converts an n-bit input into exactly one active output line.

Example: 2-to-4 Decoder



- A 3-to-8 decoder converts 3-bit input into 1 of 8 outputs.
- Used in memory addressing and instruction decoding.
- A 32-bit address decoder activates one word-line in a RAM chip.

4 Arithmetic Adders

4.1 Half Adder

Adds two 1-bit numbers (A, B).

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$$\text{Sum} = A \oplus B$$

$$\text{Cout} = A \cdot B$$

4.2 Full Adder

Adds three 1-bit numbers (A, B, Cin).

$$\text{Sum} = A \oplus B \oplus \text{Cin}$$

$$\text{Cout} = A \cdot B + B \cdot \text{Cin} + A \cdot \text{Cin}$$

Full Adder can be built using two Half Adders and one OR gate.

Ripple-Carry Adder: Cascading N full adders forms an N-bit adder (e.g. 32-bit, 64-bit) used in ALUs.

5 Instruction Set Architecture: CISC vs. RISC

5.1 The Hardware-Software Contract

- An Instruction Set Architecture (ISA) defines the interface between software (compilers) and hardware (CPU).
- Specifies instructions, register sets, addressing modes, (memory access model), without revealing implementation details.

The ISA is the contract:

Software writes instructions, and hardware executes them.

5.2 CISC vs. RISC Philosophy & Trade-offs

CISC (Complex Instruction Set Computer)

- **Philosophy**: Make hardware do more work per instruction to minimize code size and RAM usage.
- **Characteristics**:
 - Variable length instructions
 - Hundreds of specialized instructions
 - Direct memory-to-memory operations (e.g. `MULT [addr1], [addr2]`)
- **Examples**: x86 (Intel Core, AMD Ryzen)
- **Trade-off**: Complex multi-cycle hardware decoding and higher power consumption.

RISC (Reduced Instruction Set Computer)

- **Philosophy**: Keep hardware simple, fast, and uniform so every instruction executes in a single clock cycle.
- **Characteristics**:
 - Fixed 32-bit instruction length
 - Small load-store architecture (only `LOAD` and `STORE` touch memory; operations run on registers)
- **Examples**: ARM (Apple M-series, Snapdragon), RISC-V
- **Trade-off**: Slightly larger binary code sizes, but enables deep pipelining, high clock speeds, and extreme power efficiency.

Key Takeaway

- Both CISC and RISC are successful design philosophies with different trade-offs.
- The right choice depends on the application domain (high-performance desktops vs. mobile/embedded devices).

Modern Computer Architecture

Lecture 04 - Number Systems, Two's Complement & Overflow

Same Bits
Different Meaning
(Interpretation
is a Convention)

1 Positional Notation & Number Systems

- In a positional number system with base b :

$$\text{Value} = \sum_{i=0}^{n-1} d_i \times b^i$$

$d_i \rightarrow$ digit at position i
 $b \rightarrow$ base of system

- Bits in hardware are just voltage levels.
Meaning (decimal, hex, etc.) is a **convention**.

1.1 Common Number Systems

Base	Digits Used	Weights (from right)	Example	Decimal Value
Decimal (10)	0 - 9	$10^0, 10^1, 10^2, \dots$	237_{10}	237
Binary (2)	0, 1	$2^0, 2^1, 2^2, \dots$	101010_2	42
Hexadecimal (16)	0 - 9 A - F	$16^0, 16^1, 16^2, \dots$	$0x2A$	42

1 hex digit = 4 binary bits (1 nibble)

A = 10, B = 11, C = 12
D = 13, E = 14, F = 15

1.2 Conversion Examples

Binary $\rightarrow$ Decimal

$$\begin{aligned} 101010_2 \\ = 32 + 8 + 2 \\ = 42_{10} \end{aligned}$$

Decimal $\rightarrow$ Binary

$$\begin{aligned} 42 \div 2 &= 21 \dots 0 \\ 21 \div 2 &= 10 \dots 1 \\ 10 \div 2 &= 5 \dots 0 \\ 5 \div 2 &= 2 \dots 1 \\ 2 \div 2 &= 1 \dots 0 \\ 1 \div 2 &= 0 \dots 1 \end{aligned}$$

Read from bottom to top

$$\Rightarrow 42_{10} = 101010_2$$

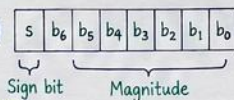
Hex $\rightarrow$ Binary

$$\begin{aligned} 0x2A \\ \downarrow \quad \uparrow \\ 2 \quad A \\ 0010 \quad 1010 \end{aligned}$$

2 Signed Integer Representations

2.1 Sign-Magnitude

- MSB is the sign bit (0 = positive, 1 = negative)
- Remaining bits store magnitude



Problems:

- Two zeros: $+0 = 00000000$, $-0 = 10000000$
- Requires separate hardware for subtraction

2.2 1's Complement

- Negation: invert all bits (bitwise NOT)
- Still has two zeros:
 - $+0 = 00000000$
 - $-0 = 11111111$
- Requires end-around carry in addition

Example (8-bit)

$$\begin{aligned} +5 &= 00000101 \\ -5 &= 11111010 \\ &\quad (\text{invert bits}) \end{aligned}$$

2.3 2's Complement (Hardware Winner)

- Standard representation in modern computers
- MSB has negative weight (-2^{n-1})
- Exactly one zero representation
- Allows addition and subtraction using the same adder

n-bit 2's Complement Range

$$-2^{n-1} \text{ to } +2^{n-1} - 1$$

(8-bit: -128 to +127)

8-bit Examples (2's Complement)

$$\begin{aligned} +127 &= 01111111 \\ -128 &= 10000000 \\ -1 &= 11111111 \\ 0 &= 00000000 \end{aligned}$$

Bit Weights (8-bit)

-128	64	32	16	8	4	2	1
b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0

3 Two's Complement Operations

3.1 Negation Algorithm (Invert +1)

To compute $-X$ in 2's complement:

$$-X = \bar{X} + 1$$

Example: Find -5 (8-bit)

- +5 in binary : 00000101
- Invert bits : 11111010
- Add 1 : 11111011

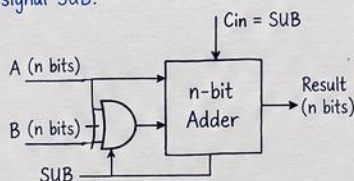
$$\Rightarrow -5 = 11111011$$

Shortcut: Invert all bits and add 1

3.2 Unified Adder for Addition & Subtraction

$$A - B = A + (\bar{B} + 1)$$

A single adder can do both using control signal SUB.



$SUB = 0 \rightarrow B$ unchanged, $Cin = 0 \rightarrow \text{Output} = A + B$
 $SUB = 1 \rightarrow B$ inverted, $Cin = 1 \rightarrow \text{Output} = A + \bar{B} + 1 = A - B$

4 Arithmetic Overflow Detection

4.1 Unsigned Carry vs. Signed Overflow

Flag	Meaning	When it occurs	Indicator
Carry Flag (C)	Unsigned overflow	Result exceeds $2^n - 1$ (in unsigned arithmetic)	$C_{out} = 1$
Overflow Flag (V)	Signed overflow	Result exceeds signed range $[-2^{n-1}, +2^{n-1} - 1]$	$C_{in, MSB} \oplus C_{out, MSB}$

4.2 Hardware Overflow Rule

$$V = C_{in, MSB} \oplus C_{out, MSB}$$

- Signed overflow occurs if carry into MSB differs from carry out of MSB.
- Happens when adding two numbers with same sign gives result with opposite sign.

Examples (8-bit)

$$\begin{aligned} 127 + 1 &= -128 \text{ (Overflow)} \\ (01111111 + 00000001) \\ &= 10000000 \\ -128 - 1 &= 127 \text{ (Overflow)} \\ (10000000 + 11111111) \\ &= 01111111 \end{aligned}$$

5 Real-World Systems & Common Pitfalls

5.1 Real Failures

- Ariane 5 Rocket Explosion (1996)**
 - 64-bit float value cast to 16-bit signed int.
 - Value exceeded 32,767 $\rightarrow$ overflow exception
 - Shut down flight computer after 37 seconds
 - Destroyed a \$500 million rocket.
- YouTube Gangnam Style (2014)**
 - Video views stored as signed 32-bit integer
 - Max = $2,147,483,647$ ($2^{31} - 1$)
 - Exceeded limit $\rightarrow$ counter became negative
 - Forced YouTube to upgrade to 64-bit integers.

5.2 Sign Extension & Casting

- When expanding a smaller signed integer to a larger register, replicate the MSB (sign bit) across new higher bits (Sign Extension).
- Zero extension is used for unsigned data types.

Example: 8-bit -5 to 32-bit (Sign Extension)

$$\begin{aligned} \text{8-bit : } &11111011 \quad (-5) \\ \text{32-bit : } &11111111111111111111111111111011 \quad (-5) \end{aligned}$$

Zero Extension (unsigned)

$$\begin{aligned} \text{8-bit : } &11111011 \\ \text{32-bit : } &00000000 \\ &00000000 \\ &00000000 \\ &11111011 \end{aligned}$$

Sequential Logic: Feedback, Latches & Memory

① Combinational vs. Sequential Logic

Combinational Logic

- Output depends only on **current inputs**
- Stateless (no memory)
- No feedback loops
- Examples: MUX, Full Adder, Decoder

$$\text{Output} = f(\text{Inputs NOW})$$

Sequential Logic

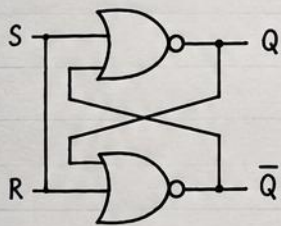
- Output depends on current inputs AND **past state**
- Stateful (has memory)
- Contains feedback loops
- Examples: Registers, Counters, Memory

$$\text{Output} = f(\text{Inputs NOW, Past State})$$

- In sequential logic, same input values can produce different outputs depending on system's prior history.
- A **clock signal** coordinates when state updates occur.

③ The SR Latch (NOR & NAND Versions)

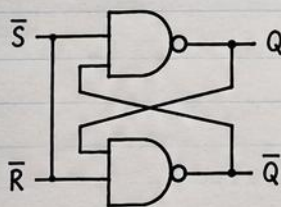
(a) NOR-Based SR Latch (Active-High)



Truth Table (NOR)

S	R	Q (next)	Operation
0	0	Q	Hold
1	0	1	Set
0	1	0	Reset
1	1	0	Forbidden!

(b) NAND-Based SR Latch (Active-Low)



Truth Table (NAND)

S̄	R̄	Q (next)	Operation
1	1	Q	Hold
0	1	1	Set
1	0	0	Reset
0	0	0	Forbidden!

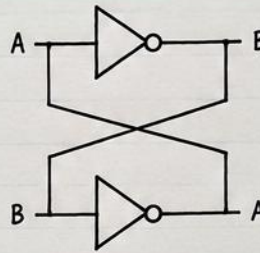
Why $S = R = 1$ (or $\bar{S} = \bar{R} = 0$) is Dangerous?

- Forces both Q and $\bar{Q}$ to the same value (breaks $Q \neq \bar{Q}$).
- If inputs return to normal simultaneously, a **race condition** occurs.
- Due to small propagation delays, final state becomes **unpredictable**.

② The Origin of Memory: Feedback Loops

- A circuit built from simple logic gates can gain memory by using **feedback loops**.
- The output signal is routed back to the input, allowing the circuit to reinforce and hold its state.
- Simplest 1-bit memory cell: **two cross-coupled inverters** (bistable element).

Cross-Coupled Inverters (Bistable Element)



Two Stable States

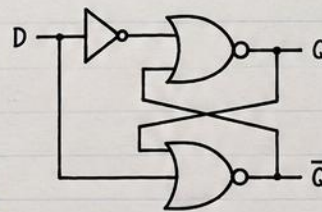
- State 1:**
 - $A = 0 \rightarrow B = 1$
 - B reinforces $A = 0$
- State 2:**
 - $A = 1 \rightarrow B = 0$
 - B reinforces $A = 1$

Limitation:

No control inputs to change the stored bit. Adding control inputs gives us a **Latch**.

④ The D Latch & Gated Enable

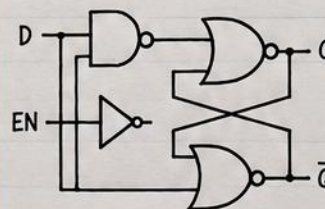
(a) D Latch (Eliminates Forbidden State)



Key Idea

- D is connected to S and $\bar{D}$ to R.
- S and R are always opposite, so $S = R = 1$ is impossible.

(b) Gated D Latch (with Enable)



State Equation

$$Q_{\text{next}} = \begin{cases} D & (\text{when } EN = 1) \\ Q & (\text{when } EN = 0) \end{cases}$$

Level Transparency

- When $EN = 1$: latch is transparent (Q follows D).
- When $EN = 0$: latch holds the last valid bit.

⑤ Practical Limits of Latches

Level Transparency Risks

- When $EN = 1$, data can pass through multiple cascaded latches in a single clock cycle (**data slippage**).
- Can corrupt internal CPU registers in pipelined architectures.
- Latches are **level-sensitive**, not edge-sensitive.

Need for Clock-Edge Triggering

- For reliable CPU registers and pipelines, we need storage elements that sample data at one precise instant in time (a clock edge).
- This leads to the Flip-Flop (covered in Lecture 06 & 07).

Key Takeaway

- Latches are useful building blocks, but for robust, synchronous designs (CPUs), edge-triggered flip-flops are preferred.

Latches → Simple memory, but level transparency causes data slippage → Use **edge-triggered Flip-Flops** for CPU registers.

Modern Computer Architecture

Lecture 06 - Latches, Flip-Flops & Timing Foundations

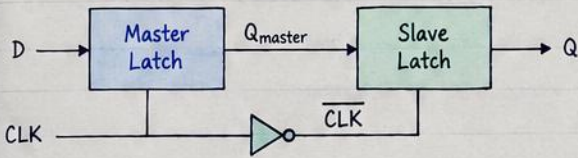
Build Logic
Build Faster

1 Key Definitions

- Latch**: Level-triggered storage element. Transparent during clock level (HIGH or LOW). Output continuously tracks input D.
- Flip-Flop**: Edge-triggered storage element. Samples input at one precise clock instant ($0 \rightarrow 1$ or $1 \rightarrow 0$). Locks output for rest of cycle.
- Setup Time (t_{setup})**: Minimum time data must be stable BEFORE the clock edge.
- Hold Time (t_{hold})**: Minimum time data must remain stable AFTER the clock edge.
- Propagation Delay (t_{pcq})**: Max time from clock edge to valid output Q.
- Contamination Delay (t_{ccq})**: Min time from clock edge before output begins to change.

3 Master-Slave D Flip-Flop

- Constructed using two D latches in series with inverted clock signals.
- Master and Slave latches are never open at the same time (two-stage airlock).

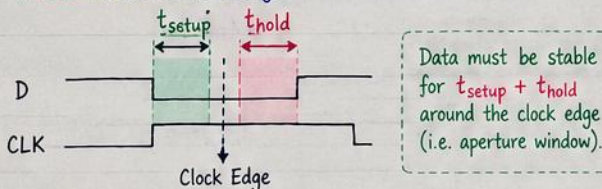


Clock Phase Hand-off:

- When $CLK = 1$: Master latch is open and tracks input D. Slave latch is closed and holds previous output Q.
- On falling edge ($1 \rightarrow 0$): Master closes and freezes Q_{master} . Slave opens and passes Q_{master} to output Q.

5 Timing Parameters: Setup & Hold

- Setup Time (t_{setup})**: Minimum time data D must be stable BEFORE the active clock edge.
- Hold Time (t_{hold})**: Minimum time data D must remain stable AFTER the active clock edge.



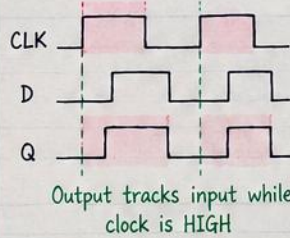
Camera Shutter Analogy:

Sampling data with a flip-flop is like taking a photo of a moving athlete. The athlete must stand still for a brief instant before and after the shutter clicks ($t_{setup} + t_{hold}$); otherwise, the image is blurred (corrupted bit).

2 Level vs Edge Triggering

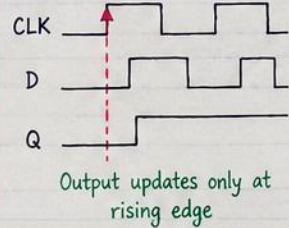
Level-Triggered (Latch)

- Active during entire clock level (e.g. $CLK = 1$).
- Output tracks input D (transparent).
- Data races can occur.



Edge-Triggered (Flip-Flop)

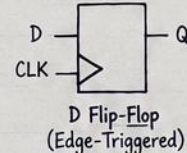
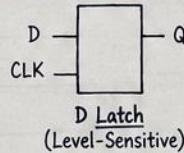
- Active only at clock transition (e.g. $0 \rightarrow 1$ rising edge).
- Captures a single snapshot of D.
- Output remains constant for rest of cycle.



4 Latch vs Flip-Flop Comparison

Feature	Latch	Flip-Flop
Triggering	Clock LEVEL (High/Low)	Clock EDGE (Rising/Falling)
Transparency	Yes (while clock is high)	No (samples at edge)
Gate Count	Fewer (~1x size)	More (~2x size)
Data Race Risk	High (requires careful timing)	Blocked (safe for pipelines)
Primary Use Case	Asynchronous buffering, SRAM	CPU Registers, PC, Pipelines

Schematic Symbols:



Triangle ($\triangleright$) on the clock input indicates edge-sensitivity.

6 Timing Violations & Propagation Delays

Setup/Hold Violations:

- If data changes inside the setup or hold aperture window, a **timing violation** occurs.
- Flip-flop may capture incorrect value or enter a dangerous state known as **Metastability** (detailed in Lecture 07).

Propagation Delays:

- t_{pcq} (Propagation Delay Clock-to-Q): Maximum time after clock edge until output Q is stable.
- t_{ccq} (Contamination Delay Clock-to-Q): Minimum time after clock edge before output Q begins changing.

Maximum Clock Frequency Constraint:

$$T_c \geq t_{pcq} + t_{pd,combinational,max} + t_{setup}$$

$$\text{or } f_{max} = \frac{1}{t_{pcq} + t_{pd,combinational,max} + t_{setup}}$$

t_{pd} = combinational delay

Modern Computer Architecture

Metastability, Synchronizers & Static Timing Analysis

Lecture 07

- Reliable ✓
- Fast ✓
- Scalable ✓

① Metastability - Basics

- When a flip-flop experiences a **setup or hold time violation**, its output does not resolve immediately to 0 or 1, but stays at an intermediate voltage for an unpredictable time.

"Metastability is a temporary analog state in a digital system."

• Ball-on-a-Hill Model

- Flip-flop is like a ball on top of a hill.
- If disturbed near the peak, it can take an unpredictable time to roll down to 0 or 1.



• System State Divergence Hazard

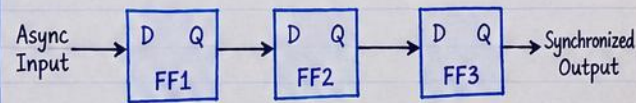
- If a metastable signal feeds multiple logic paths, different gates may interpret it differently (one as 0, another as 1) $\Rightarrow$ inconsistent system state.
 $\Rightarrow$ Can crash OS, corrupt memory or cause serious failures.

② Asynchronous Clock Domain Crossing

- Occurs when a signal moves between independent clock domains (e.g., external I/O, PCIe devices, network cards, user buttons).
- External signal changes at random times relative to CPU clock $\Rightarrow$ setup/hold violations will happen.
- Metastability cannot be completely eliminated, but its effect can be minimized.

Synchronizer Chains (2-FF & 3-FF)

- Use a chain of flip-flops (clocked by destination clock) to safely bring asynchronous inputs into the system.
- If FF1 goes metastable, it gets a full clock cycle (T_{clk}) to settle before FF2 samples it.
- Each additional flip-flop exponentially reduces failure probability.



- $\uparrow$ 2-FF is enough for most cases.
- $\uparrow$ 3-FF is used for ultra-critical systems.

③ Mean Time Between Failures (MTBF)

- Reliability of a synchronizer is quantified by MTBF.
- MTBF increases exponentially with the available resolution time t_r .

$$MTBF = \frac{e^{(t_r/\tau)}}{T_0 \cdot f \cdot f_D}$$

- t_r : available settling time ($\approx T_{clk}$ per stage)
- τ : technology-dependent time constant (≈ 10 - 100 ps)
- T_0 : device-specific constant ($\approx 10^{-9}$ - 10^{-12})
- f : destination clock frequency
- f_D : data transition rate (asynchronous input frequency)

Key Insight:

- Because t_r is in the exponent, adding a second flip-flop increases MTBF from a few seconds to billions of years.
- In practice, a 2-FF synchronizer is sufficient for most applications.

④ Real-World Clock Speeds & Critical Path

Clock Period Calculation

- A CPU clock frequency determines the allowable time for combinational logic between registers.

$$T_{clk} = \frac{1}{f}$$

Example:

$$f = 3.5 \text{ GHz}$$

$$\Rightarrow T_{clk} = \frac{1}{3.5 \times 10^9} \approx 0.285 \text{ ns} \quad (285 \text{ ps})$$

Critical Path

- The longest and slowest combinational logic path between any two register flip-flops.
- This single path determines the maximum operating frequency of the chip.
- All other paths must also meet timing constraints.

Timing Constraint (Setup):

$$T_{clk} \geq t_{pcq} + t_{logic,max} + t_{setup} + t_{skew}$$

- t_{pcq} : clk to Q delay
- t_{logic} : max comb. delay
- t_{setup} : setup time
- t_{skew} : clock skew

⑤ Overclocking Hazards

- Overclocking forces T_{clk} below factory specifications.
- Timing safety margin shrinks $\Rightarrow$ higher chance of setup violations.
- Thermal expansion, voltage drops and process variations increase gate delays.
- Can lead to silent data corruption (hard to detect).

⚠ Overclocking may give higher performance, but at the cost of reliability and stability.

⑥ Timing Verification & STA

- Before fabrication, EDA tools perform **Static Timing Analysis (STA)** to verify that every path meets setup and hold constraints across all operating conditions (voltage, temperature, process).

Industry Tool: Synopsys PrimeTime

- Checks setup & hold constraints.
- Verifies timing across all paths.
- Considers PVT variations (Process, Voltage, Temperature).
- Ensures chip meets timing at different operating conditions.

Other Verification Techniques

- Circuit simulation (SPICE)
- Formal verification
- On-chip monitoring
- Design margining for temperature and voltage variations

Goal:

- Ship reliable hardware that works across all conditions for its lifetime.

Modern Computer Architecture – Lecture 08

Flip-Flop Types, Registers & Digital Counters

Build
Store
Count
Repeat!

1. The Flip-Flop Toolkit (SR, D, JK, T)

1.1 Characteristic Tables, Equations & Circuits

Type	Symbol	Characteristic Table			Equation
SR		S	R	Q(n+1)	$Q(n+1) = S + \overline{R}Q(n)$ Constraint: $S \cdot R = 0$
		0	0	Q(n) (Hold)	
		0	1	0 (Reset)	
		1	0	1 (Set)	
		1	1	Invalid	
D		D	Q(n+1)		$Q(n+1) = D$ (Data copy)
		0	0		
		1	1		
JK		J	K	Q(n+1)	$Q(n+1) = J\overline{Q}(n) + \overline{K}Q(n)$ (Universal flip-flop)
		0	0	Q(n)	
		0	1	0	
		1	0	1	
		1	1	$\overline{Q}(n)$ (Toggle)	
T		T	Q(n+1)		$Q(n+1) = T \oplus Q(n)$ (Counting engine)
		0	Q(n)		
		1	$\overline{Q}(n)$ (Toggle)		

1.2 JK Flip-Flop: Eliminating Forbidden State

- SR latch has illegal state $S = R = 1$.
- JK flip-flop removes this problem.
- When $J = 1$ and $K = 1$, output toggles ($Q \rightarrow \overline{Q}$) on every clock edge (no race condition).
- Hence JK is called universal flip-flop (can implement SR, D and T).

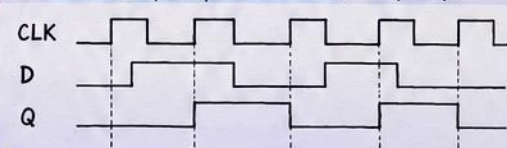
1.3 T Flip-Flop: Toggle & Counting Engine

- $T' = 0 \rightarrow Q(n+1) = Q(n)$ (Hold)
- $T = 1 \rightarrow Q(n+1) = \overline{Q}(n)$ (Toggle)

Frequency Division Rule :

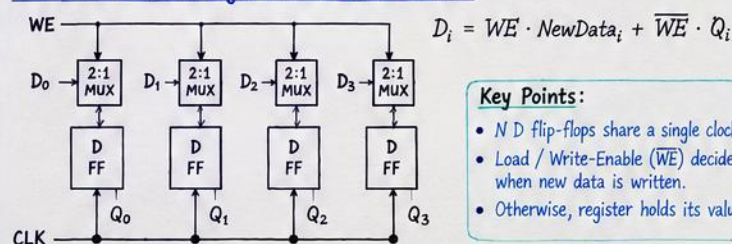
- A T flip-flop with $T = 1$ divides input clock frequency by 2.
- For N cascaded stages : $f_{out} = \frac{f_{clk}}{2^N}$

1.4 Clocked Flip-Flop Waveforms (D Flip-Flop)



2. Multi-Bit Registers & Shift Registers

2.1 Parallel Load Register with Clock Enable



Key Points:

- N D flip-flops share a single clock.
- Load / Write-Enable ($\overline{WE}$) decides when new data is written.
- Otherwise, register holds its value.

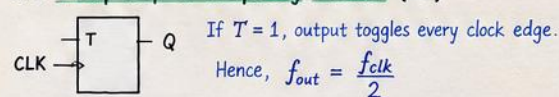
2.2 Shift Registers

Connecting output of flip-flop k to input of $k+1$ moves bits one position on each clock pulse.

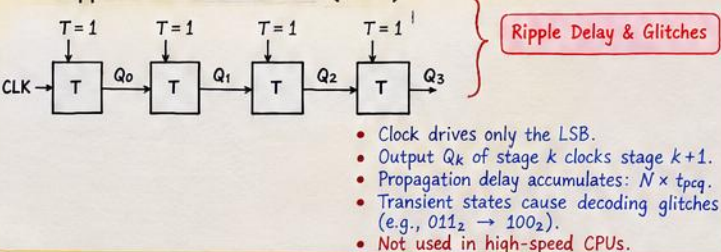
Type	Diagram	Use Case
SISO		Serial-In Serial-Out (Data delay, UART)
SIPO		Serial-In Parallel-Out (Serial to 8-bit bus)
PISO		Parallel-In Serial-Out (Parallel to serial)
PIPO		Parallel-In Parallel-Out (CPU pipeline register)

3. Asynchronous (Ripple) Counters

3.1 T Flip-Flop as Frequency Divider ($\div 2$)



3.2 Ripple Counter Structure (4-bit)



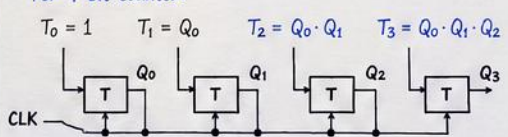
- Clock drives only the LSB.
- Output Q_k of stage k clocks stage $k+1$.
- Propagation delay accumulates: $N \times t_{pg}$.
- Transient states cause decoding glitches (e.g., $011_2 \rightarrow 100_2$).
- Not used in high-speed CPUs.

4. Synchronous Counters

4.1 Shared Clock Architecture & Lookahead Gates

- All flip-flops receive the same clock simultaneously.
- Eliminates ripple delays and decoding glitches.
- Each T input is generated using lookahead AND gates.

For 4-bit counter:



4.2 Up/Down & Modulo-N Counter Design

- Use control MUXes to switch between counting up and down.
- Add clear/reset logic when reaching a target count.
- Example: Modulo-10 (BCD) counter (counts 0-9, then reset).

5. Practical Applications & Trade-offs

5.1 Real-World Applications

Application	Description
Program Counter (PC)	Holds address of next instruction, increments every cycle.
Baud Rate Generators	Divide high-frequency clock to get UART rates (e.g., 115200 bps).
PWM (Pulse Width Mod.)	Controls motor speed, LED brightness (compare counter with threshold).
Timers / Event Counters	Used in microcontrollers, real-time systems.
Frequency Dividers	Generate lower-frequency clocks from high-frequency sources.

5.2 Excitation Tables & Custom Counter Design

- For non-sequential counters (e.g., $0 \rightarrow 2 \rightarrow 5 \rightarrow 1 \rightarrow 0$), use an excitation table.
- Determine required inputs (D, JK or T) for each transition $Q(n) \rightarrow Q(n+1)$.
- Implement using minimal gates / K-maps.

Design Recipe:

- Define state sequence.
- Create state / excitation table.
- Derive input equations (simplify).
- Implement and verify with timing diagrams.

Key Takeaways

- ✓ D FF $\rightarrow$ storage
- ✓ JK FF $\rightarrow$ universal
- ✓ T FF $\rightarrow$ counting
- ✓ Registers $\rightarrow$ multi-bit storage
- ✓ Ripple counters $\rightarrow$ simple but slow
- ✓ Synchronous counters $\rightarrow$ fast and glitch-free
- ✓ Counters are everywhere in real systems!

Modern Computer Architecture

Lecture 09 - Register Files & Finite State Machines (FSMs)

Registers store data
FSMs control behaviour
→ Heart of CPU!

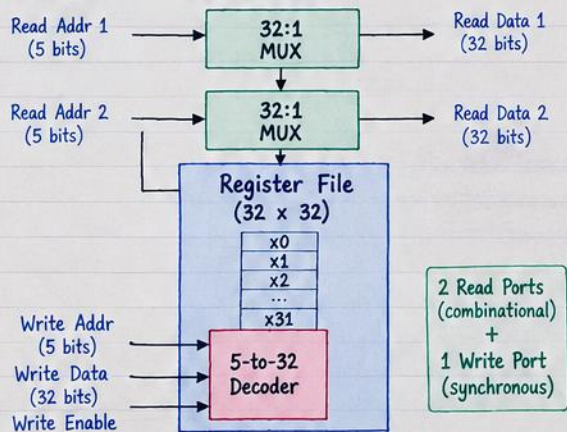
1. Register Files: Multi-Port CPU Storage

- A typical RISC instruction (e.g. `add r1, r2, r3`) needs two source registers and one destination register simultaneously in a single clock cycle.
- Individual registers cannot provide multi-port access.
- A **Register File** groups N registers (e.g., 32 registers of 32 bits in RISC-V/MIPS) into one block with **2 Read Ports + 1 Write Port**.

Typical RISC Register File (e.g. RISC-V)

- 32 registers (x0 - x31)
- Each register: 32 bits
- 2 independent read ports
- 1 write port
- Supports two-operand ALU execution per cycle.

1.2 Register File Block Diagram



1.3 Write Port (Decoder + Write-Enable)

- 5-bit write address feeds a 5-to-32 address decoder.
- Decoder asserts exactly 1 of the 32 clock-enable lines (one-hot).
- If **WE** = 1, the selected register captures **Wdata** on the clock edge.
- Unselected registers keep their current bits.

1.4 Read Ports (Dual 32:1 MUX Trees)

- Each read port has a 32:1 multiplexer connected to outputs of all 32 registers.
- Two separate MUX trees allow reading two independent registers simultaneously.

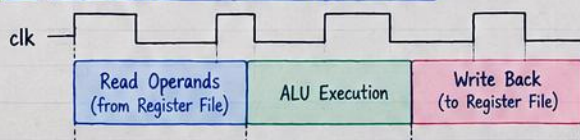
2. Register File Timing Asymmetry

Operation	Type	Clock Required?	When Does it Happen?
Read	Combinational	No	Immediately (after gate delay)
Write	Synchronous	Yes	On active clock edge (when WE = 1)

Key Points:

- Reads are **combinational** → supplying a read address immediately routes data through MUX tree.
- Writes are synchronous** → data is captured only on the clock edge when WE = 1.

2.2 Single-Cycle Read-Before-Write Protocol



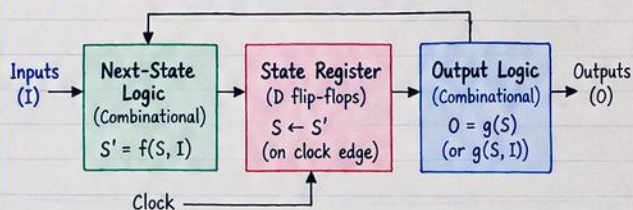
At the start of the cycle, CPU reads operands → ALU processes during cycle → at clock edge, result is written back.

3. Fundamentals of Finite State Machines (FSMs)

3.1 Definition & Elements

- A **Finite State Machine (FSM)** is a mathematical model of sequential decision-making logic.
- It consists of:
 - States (S)**: finite set of discrete conditions (e.g., FETCH, DECODE)
 - Inputs (I)**: external conditions that influence transitions
 - Transitions**: rules governing movement from current state to next state
 - Outputs (O)**: control signals generated by the machine

3.2 3-Part Structural Model of an FSM



4. Moore vs. Mealy Machine Architectures

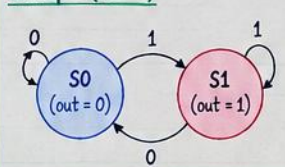
4.1 Moore Machine (Output = f(State))

- Outputs depend only on current state.
- Output = f(State)
- Outputs written **inside** state circles.
- Outputs change only on clock edges (glitch-free).
- May require more states.

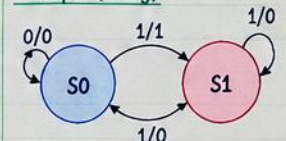
4.2 Mealy Machine (Output = f(State, Input))

- Outputs depend on current state and input.
- Output = f(State, Inputs)
- Outputs written on transition arrows (input/output).
- Can react to input changes within same clock cycle.
- Often requires fewer states, but outputs can glitch if inputs wobble.

Example (Moore)



Example (Mealy)



5. FSM Synthesis (Design Procedure)

- Understand Problem** → identify states, inputs, outputs.
- Draw State Diagram** → sketch states and transitions.
- Construct State Table** → current state + input → next state + output.
- Encode States** → assign binary codes (e.g., S0 = 00, S1 = 01).
- Derive Logic Equations** → minimize using K-maps.
- Build Hardware** → D flip-flops + combinational gates.

6. CPU Control Unit: Datapath Orchestration

- The **Control Unit** is a giant FSM.
- It receives status **flags** from datapath (e.g., ALU result zero, overflow).
- It generates **control signals** (register write-enable, MUX select codes, ALU operation selection) to orchestrate instruction execution.

Examples of Control Signals:

- Register Write Enable (RegWrite)
- ALU Operation Select (ALUOp)
- MUX Select Signals (e.g., MemToReg, ALUSrc)
- Memory Read/Write (MemRead, MemWrite)
- Next State Selection based on instruction type and flags

Control Unit (FSM) coordinates the entire datapath, ensuring each instruction goes through the correct sequence of micro-operations.

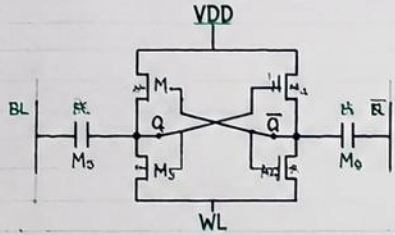
Modern Computer Architecture

Faster Memory
Smarter Systems

Lecture 10 - Memory Technologies : SRAM vs. DRAM & Architecture

① Static RAM (SRAM) & 6T Cell

1.1 6-Transistor (6T) SRAM Cell



- 4 transistors (M1-M4) → two cross-coupled inverters (bistable latch).
- 2 transistors (M5, M6) → access pass-gates controlled by Word Line (WL)
- Stores 1 bit using active feedback.

1.2 Self-Reinforcing Latch & Non-Destructive Reads

- Cross-coupled inverters powered by VDD, self-correct any leakage.
- Reading is **non-destructive** (bit remains intact).
- **No refresh needed** (as long as power is supplied).

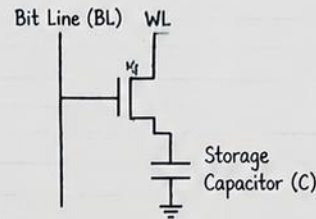
1.3 CPU Cache Applications

- Uses 6 transistors per bit → large silicon area & expensive.
- Used in speed-critical CPU caches (L1, L2, L3).
- Provides **sub-nanosecond** access times (~1-2 ns).

Used for
L1 / L2 / L3
Caches

② Dynamic RAM (DRAM) & 1T1C Cell

2.1 1-Transistor 1-Capacitor (1T1C) Cell



- 1 transistor (access) → connects capacitor to bit line when WL is high.
- 1 capacitor → stores charge (1 = charged, 0 = discharged).
- Much smaller cell size than SRAM (~6x higher bit density).

2.2 Charge Leakage & Refresh Overhead

- Capacitor charge **leaks away** through silicon substrate in milliseconds.
- Requires **periodic refresh** (every ~64 ms) by memory controller.
- During refresh, memory accesses are briefly stalled.

2.3 Destructive Reads & Sense Amplifiers

- Reading drains the capacitor (**destructive read**).
- Produces a tiny **voltage** change on the bit line.
- Sense amplifiers detect and amplify the signal.
- The bit is immediately written back into the cell.

Slower, but
extremely dense
and affordable
→ used for
Main Memory
(DDR4 / DDR5)

③ SRAM vs. DRAM Comprehensive Matrix

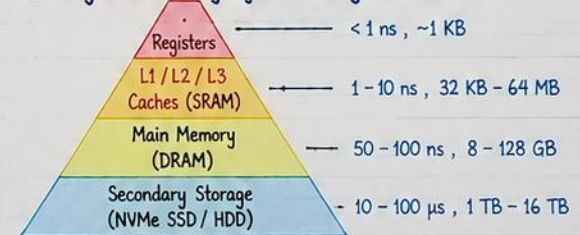
Feature	SRAM (Static RAM)	DRAM (Dynamic RAM)
Cell Construction	6 Transistors (6T)	1 Transistor + 1 Capacitor (1T1C)
Bit Density	Low (Bulky, 6T)	High (~6x more bits/mm ²)
Access Latency	~1 - 2 ns	~50 - 100 ns
Refresh Needed?	No (Holds while powered)	Yes (every ~64 ms)
Read Nature	Non-destructive	Destructive (requires write-back)
Cost per Bit	High (Expensive)	Low (Very cheap per GB)
Primary Role	CPU Caches (L1, L2, L3)	Main System RAM (DDR4/DDR5)

Physical Size Comparison



④ The Memory Hierarchy & Memory Wall

4.1 Layered Memory Pyramid (Registers to Disk)



4.2 The Memory Wall

- CPU clock speeds improved by ~50% per year (last 40 years).
- DRAM latency improved by only ~7% per year.
- This growing gap is called the **Memory Wall**.
- High-performance CPUs spend many cycles waiting for DRAM unless caches hide the latency.

CPU: GHz (fast)
DRAM: ~50-100 ns (slow)
→ Huge gap!

⑤ System Organization & von Neumann Principle

5.1 Stored-Program Concept ("Programs ARE Data")

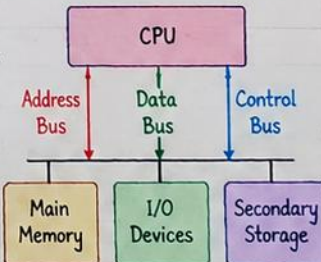
- In **von Neumann** architecture, instruction code and user data reside in the same memory space.
- Programs are just binary data stored in memory.
- Operating systems load programs from disk to RAM and jump to their entry address.
- Compilers generate machine code as data output.
- Just-In-Time (JIT) engines (e.g., in browsers) generate machine code at runtime.

"A program is just another form of data."

5.2 Bus Architectures & CPU Interconnects

- CPU communicates with memory and I/O using three system buses:
- **Address Bus**: Specifies the memory location (unidirectional).
- **Data Bus**: Carries data words (bidirectional).
- **Control Bus**: Carries control signals (Read, Write, Interrupt, etc.).

These buses form the backbone of the von Neumann system, connecting CPU, memory, and I/O devices.

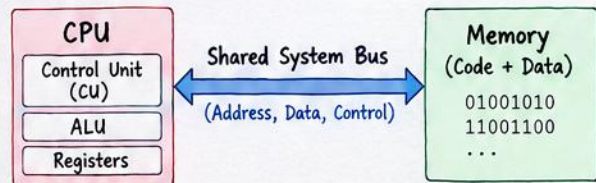


Modern Computer Architecture

Lecture 11 – Computer Organization: Von Neumann vs. Harvard Architecture & The Bottleneck

1. Von Neumann Architecture (1945)

- Proposed by **John von Neumann** (EDVAC report, 1945)
- Stored-program concept** → program instructions and data stored in the same memory.
- Single unified memory and single shared bus.
- Simple, universal, low cost.



1.1 Unified Memory Space

Address	Contents (32-bit)	Interpretation
0x00000000	0x8D210040	Instruction: lw r1, 0x40(r0)
0x00000004	0x00221820	Instruction: add r3, r1, r2
0x00000008	0xAD230044	Instruction: sw r3, 0x44(r0)
0x00000040	0x0000002A	Data: Integer value 42
0x00000044	0x00000000	Data: Output buffer

1.2 Four Classic Components

- Control Unit (CU)** – fetch, decode, generate control signals
- ALU** – performs arithmetic & logical operations
- Unified Memory** – stores both code and data
- Input/Output (I/O)** – interfaces with external devices

1.3 "Programs ARE Data" – Uses & Risks

- Enables: compilers, OS loaders, JIT compilation
- Security risk**: buffer overflow, code injection
- Mitigation: **NX/DEP** (Never Execute / Data Execution Prevention)

4. Modified Harvard Architecture (Modern Hybrid)

- Unified main memory (DRAM) outside CPU (von Neumann style).
- Split L1 Instruction and L1 Data caches inside CPU (Harvard style).
- L1-I and L1-D allow simultaneous fetch and data access.
- L2 / L3 caches unified, connected to unified DRAM.
- Hardware cache coherence maintains consistency (important for self-modifying code / JIT).
- Used in modern PCs, mobile and servers (x86-64, ARM, Apple M-series).

2. The Von Neumann Bottleneck

- Single shared bus → only **one memory transaction** at a time.
- CPU cannot fetch instruction and access data simultaneously.
- Causes CPU stalls, limiting Instruction Per Cycle (IPC).
- Term coined by John Backus (1977).
- CPU speed ↑ (~50% per year) much faster than DRAM speed (~7% per year) → widening gap.

Example: LOAD R1, [0x1004]

- Cycle 1:** Fetch instruction from memory (bus occupied).
 - Cycle 2:** Read data from memory (bus occupied).
- Needs minimum **2 cycles**.

Result:

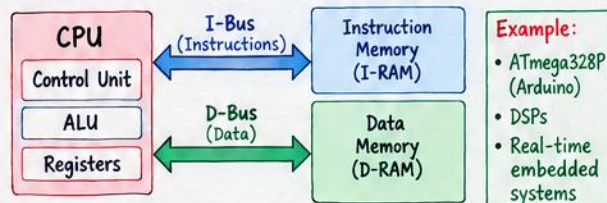
CPU often idle waiting for memory transfers.

Architectural Responses

- Caching (SRAM)** – small, fast on-chip caches.
- Wider & Faster Buses** – 16 → 32 → 64 → 128-bit, multi-channel memory.
- Harvard-style split buses** – separate paths for instructions and data.

3. Harvard Architecture

- Separate instruction and data memories.**
- Two independent buses** (I-Bus and D-Bus).
- Can fetch instruction and access data **simultaneously** (1 cycle).
- Higher throughput, deterministic timing.
- Used in DSPs, microcontrollers, real-time systems.



Example:

- ATmega328P (Arduino)
- DSPs
- Real-time embedded systems

5. Comparison Matrix

Criterion	Von Neumann	Harvard	Modified Harvard
Memory Units	1 Unified (Code + Data)	2 Separate (I-RAM & D-RAM)	Unified DRAM + Split L1
Bus Structure	1 Shared Bus	2 Independent Buses	Split L1 buses; unified outer
Cycle Efficiency	2 Cycles (sequential)	1 Cycle (simultaneous)	1 Cycle at L1 level
Memory Throughput	Bottlenecked	Up to 2x higher	Near 2x via L1
Hardware Complexity	Low	High	Moderate to High
Programming Flexibility	High	Low	High

6. Memory Bandwidth Example (1 GHz CPU)

Fetch one 32-bit instruction and read one 32-bit data word per cycle.

- Data per cycle = 32 + 32 = 64 bits/cycle
- Total bandwidth = $64 \times 10^9 \text{ bits/s} = 64 \text{ Gbit/s} = 8 \text{ GB/s}$
- Von Neumann (single bus)**: needs 8 GB/s on one bus. If bus = 4 GB/s → saturates → IPC ≈ 0.5
- Harvard (two buses)**: I-Bus = 4 GB/s, D-Bus = 4 GB/s (both concurrent) → Max IPC = 1.0

7. Real-World Examples

Processor / Platform	Architecture Type
ATmega328P (Arduino)	True Harvard
ARM Cortex-M4	Modified Harvard
x86-64 (Intel / AMD)	Modified Harvard
Apple Silicon (M1 / M2)	Modified Harvard

8. Key Takeaways

- Von Neumann: single memory, single bus. Simple but limited by serial access.
- Bottleneck**: CPU stalls due to shared bus.
- Harvard**: separate memories and buses. 1 cycle fetch + data.
- Modified Harvard**: unified DRAM with split L1 caches. Universal design.

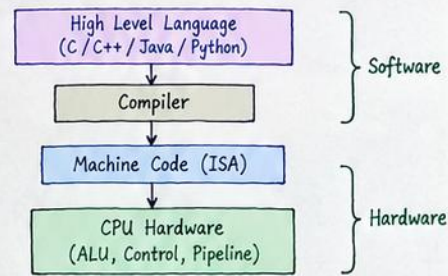
9. Exam Pitfalls

- Harvard ≠ infinite bandwidth.
- Modern PCs are not pure Harvard.
- Remember: Von Neumann = 1, Harvard = 2.
- Bus width ≠ architecture type.
- Harvard incurs higher hardware cost.

ISA Design Principles : RISC vs CISC Architecture & Hardware/Software Interface

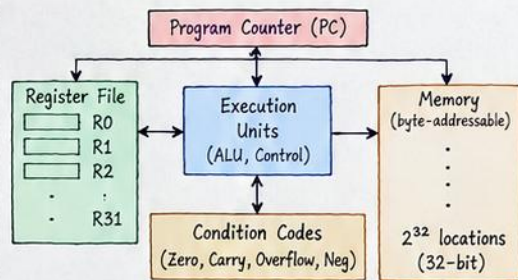
① The ISA Abstraction & Hardware/Software Interface

- Instruction Set Architecture (ISA) is the contract between software (compiler) and hardware (CPU).
- High level languages (C/C++/Java/Python) → compiled to machine instructions (ISA).
- Defines : instructions, registers, addressing modes, memory model and execution semantics.
- Hardware implements execution units (ALU, control logic, pipelines) to execute these instructions.



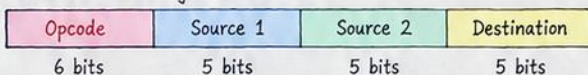
② State Elements & Program Counter

- ISA exposes a formal architectural state to the programmer.
- Program Counter (PC)** : holds address of current/next instr.
- Register File** : small, fast storage locations.
 - (e.g. 32 registers of 32 bits each in MIPS / RV32).
- Memory Address Space** : linear, byte-addressable
 - (2^{32} bytes for 32-bit ISA).
- Condition Codes / Flags** : status bits (Zero, Carry, Overflow, Negative etc.).



③ Instruction Semantics & Machine Code

- An instruction specifies:
 - Opcode** : what operation to perform.
 - Source Operands** : where inputs come from.
 - Destination** : where result is stored.
- Encoded into binary machine code.



④ Hardware Decoder Complexity

- Control logic must decode incoming binary instructions.
- Instruction complexity dictates decoder size:
 - Fixed-field encodings → minimal wiring.
 - Variable-byte encodings → multi-cycle decoding networks.

⑤ CISC Philosophy (Complex Instruction Set Computing)

- Emerging in 1970s-1980s when memory was small & expensive.
- Rich, multi-byte, variable length** instructions.
- Single instruction can perform memory fetch, ALU calculation and memory writeback.
- Examples : x86, VAX.

Direct Memory-to-Memory Operations

```

ADD [EBX + 4], EAX
Memory[EBX + 4] ← Memory[EBX + 4] + EAX
  
```

Key Features

- Variable length (1 to 15 bytes)
- Complex instructions
- Memory-to-memory operations
- High code density
- Fewer registers (8-16)
- Complex hardware decoder
- Harder to pipeline cleanly

Why CISC?

- Smaller code size
- Efficient use of memory
- Closer to high-level language statements

Challenges

- Variable instruction length
- Complex multi-stage decoder
- Difficult pipelining
- Higher CPI (2-10+ cycles)

⑥ RISC Philosophy (Reduced Instruction Set Computing)

- Developed from IBM, Berkeley and Stanford research.
- Uniform, fixed-length (typically 32-bit) instructions.
- Examples : MIPS, ARM, RISC-V.

Load/Store Architecture (Key Rule)

Memory access is strictly restricted to Load (lw) and Store (sw) instructions. All arithmetic and logical instructions operate only on registers.

Typical RISC Sequence

```

lw    t0, 0(s0)
add   t1, t0, t2
sw    t1, 0(s0)
  
```

Key Features

- Fixed length (32 bits)
- Load/Store only
- Many registers (32+)
- Simple, single-cycle decoder
- Ideal for pipelining
- Low CPI (~1.0)
- More instructions (compiler burden)

⑦ CISC vs RISC Comparison

Feature	CISC Architecture	RISC Architecture
Instruction Length	Variable (1 to 15 bytes)	Fixed (32 bits / 4 bytes)
Memory Access	Combined with arithmetic	Load/Store only
Registers	Fewer (8-16 registers)	Many (32+ registers)
Clock Cycles (CPI)	High & variable (2-10+ cycles)	Low (~1.0 cycle per stage)
Decoder Hardware	Large, complex microcode ROM	Small, fast hardwired logic
Code Size	Smaller (high density)	Larger (more instructions)
Pipelining	Harder (variable length)	Easier (fixed length)
Compiler vs Hardware	More complexity in hardware	More complexity in compiler

⑧ Hardware vs Compiler Trade-off

- CISC** : Places complexity into complex hardware decoders and microcode ROMs.
- RISC** : Shifts complexity into optimizing compiler backends.
- RISC minimizes memory transfer bytes, while maximizing clock frequency (f_{max}) and pipeline throughput.

⑨ Modern Convergence & RISC-V Open ISA

- Modern x86 (Intel Core, AMD Ryzen) : Decode CISC instructions into fixed RISC-like micro-operations (μ -ops) for superscalar execution.
- ARM & Apple Silicon** : Use clean 64-bit RISC ISAs (ARMv8/v9) for high performance per watt.
- RISC-V** : Open-source, royalty-free ISA developed at UC Berkeley. Minimal integer base (RV32I / RV64I) with modular extensions.

RISC-V Extensions

- M** : Integer Multiply & Divide
- A** : Atomic Memory Instructions
- F** : Single-Precision Floating-Point
- D** : Double-Precision Floating-Point
- C** : Compressed 16-bit Instructions (CISC-like code density!)

Self-Assessment

- Defining memory access rule of RISC? → Load/Store only.
- How do modern x86 chips work? → Decode to RISC-like μ -ops.
- What does RV32IMAFD mean? → 32-bit base integer ISA with M, A, F, D extensions.

① MIPS32 Instruction Encoding Formats

- R-Type** (Register-Register ALU) : Used for add, sub, and, or, slt.
- I-Type** (Immediate & Memory) : Used for addi, lw, sw, beq, bne.
- J-Type** (Unconditional Jump) : Used for j and jal.

R-Type :

opcode	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
31...26	25...21	20...16	15...11	10...6	5...0

- opcode = 0 (000000)
- rs = source 1
- rt = source 2
- rd = destination
- shamt = shift amount
- funct = selects operation

I-Type :

opcode	rs	rt	immediate / offset
6 bits	5 bits	5 bits	16 bits
31...26	25...21	20...16	15...0

- used for addi, lw, sw, beq, bne
- 16-bit immediate is sign-extended to 32 bits (for arithmetic & memory).

J-Type :

opcode	target address
6 bits	26 bits
31...26	25...0

- used for j and jal
- 16-bit target shifted left by 2 bits (word aligned)
- final address = {PC[31:28], target, 00}

② MIPS32 Register Conventions

Register Name	Number	Conventional Role
\$zero	0	Hardwired Constant Zero (0x00000000)
\$v0 - \$v1	2 - 3	Function Return Values / Results
\$a0 - \$a3	4 - 7	Function Arguments 1 through 4
\$t0 - \$t7	8 - 15	Temporaries (Caller-Saved)
\$s0 - \$s7	16 - 23	Saved Values (Callee-Saved)
\$t8 - \$t9	24 - 25	Temporaries (Caller-Saved)
\$gp	28	Global Pointer
\$sp	29	Stack Pointer (stack grows downwards)
\$fp	30	Frame Pointer
\$ra	31	Return Address (written by jal)

Special Notes

- \$zero always returns 0, writes are discarded.
- li \$t0, 5 → addi \$t0, \$zero, 5
- move \$t0, \$t1 → add \$t0, \$t1, \$zero
- nop → sll \$0, \$0, 0

Caller-Saved vs Callee-Saved

- Caller-Saved (not preserved):**
\$t0-\$t9, \$a0-\$a3, \$v0-\$v1
- Callee-Saved (preserved):**
\$s0-\$s7, \$sp, \$fp, \$ra, \$gp
- Caller must save \$t registers if needed after a function call.
- Callee must save/restore \$s registers if used inside the function.

③ R-Type Format (Register ALU)

opcode	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits
31...26	25...21	20...16	15...11	10...6	5...0

- opcode = 000000 for all R-type instructions.
- funct field specifies the operation (e.g. add, sub, and, or, slt).
- shamt used for shift instructions (0 for add/sub).

Example

Encode: add \$t0, \$s1, \$s2

- \$s1 = 17 (10001), \$s2 = 18 (10010), \$t0 = 8 (01000)
- opcode = 0 (000000), shamt = 0 (00000), funct = 32 (100000)
- Binary: 000000 10001 10010 01000 00000 100000
- Hex: 0x02324020

④ I-Type Format (Immediate & Memory)

opcode	rs	rt	immediate / offset
6 bits	5 bits	5 bits	16 bits
31...26	25...21	20...16	15...0

- used for addi, lw, sw, beq, bne.
- 16-bit immediate is sign-extended to 32 bits.
- Address (for lw/sw) = Reg[rs] + SignExtend(immediate).

Example

lw \$t0, 8(\$s0)

- opcode (lw) = 35 (100011)
- rs = \$s0 = 16 (10000)
- rt = \$t0 = 8 (01000)
- immediate = 8 (0000000000001000)

⑤ J-Type Format (Unconditional Jump)

opcode	target address
6 bits	26 bits
31...26	25...0

- used for j and jal.
- 26-bit target shifted left by 2 bits (word aligned).
- final address = {PC[31:28], target, 00}.

⑥ RISC-V Modular ISA Architecture

- RV32I** : Base integer instruction set (40 instructions).
- +M** : Multiply / Divide extension.
- +A** : Atomic memory instructions.
- +F / +D** : Single and double precision floating point.
- +C** : Compressed 16-bit instructions (for smaller code size).

MIPS vs RISC-V (Key Points)

- RISC-V fixes rs1 and rs2 in same bit positions across formats.
- Simpler, more regular instruction formats.
- Modular design with optional extensions.
- x0 in RISC-V is same as \$zero in MIPS (hardwired 0).

Modern Computer Architecture

Lecture 14 - Control Flow Translation, Relational Idioms & Stack Frame Calling Conventions

1. Key Points (Control Flow Translation)

- Assembly is flat → structured code (if/else, while, for) must be converted to labels and jumps.
- Branch AWAY rule** → in if(cond){ body }, branch away to skip body when condition is **FALSE**.
- MIPS has no blt, bgt, ble, bge → use **slt** + beq/bne.
- Function calls use jal (save return addr in \$ra) and return with jr \$ra.
- Non-leaf & recursive functions must save \$ra and saved registers (\$s0-\$s7) on stack.
- Stack grows downward (high → low addresses).

2. if/else Translation (Branch-Away Rule)

C code:

```
if (i == j)
    f = g + h;
else
    f = g - h;
```

MIPS assembly:

```
bne $s3, $s4, Else # if i!=j
add $s0, $s1, $s2 # f = g + h
j Exit # skip else
Else:
sub $s0, $s1, $s2 # f = g - h
Exit:
```

Test the **inverse condition** and **branch AWAY** to the else (or exit) when it is FALSE.

3. Relational Ordering & slt Idiom

- MIPS has no blt, bgt, ble, bge instructions.
- Use slt (set less than) to perform comparison, then branch on the result (0 or 1).
- slt \$t0, \$s1, \$s2 → \$t0 = 1 if \$s1 < \$s2, else 0.

Test	Assembly sequence	Meaning
a < b	slt \$t0, \$a, \$b bne \$t0, \$zero, L	branch if a < b
a ≥ b	slt \$t0, \$a, \$b beq \$t0, \$zero, L	branch if a ≥ b
a > b	slt \$t0, \$b, \$a bne \$t0, \$zero, L	branch if a > b
a ≤ b	slt \$t0, \$b, \$a beq \$t0, \$zero, L	branch if a ≤ b

Example: if (a < b) max = b;

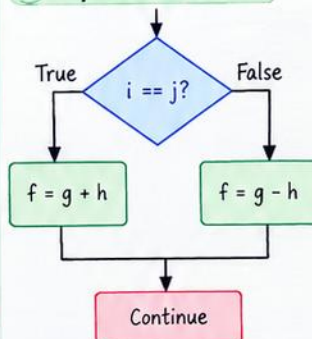
C code:

```
if (a < b) max = b;
```

MIPS:

```
slt $t0, $s0, $s1 # t0 = 1 if a < b
beq $t0, $zero, Skip
move $s2, $s1 # max = b
Skip:
```

4. if/else Flowchart



5. Loop Translation (while)

C code:

```
while (save[i] == k)
    i += 1;
```

MIPS assembly:

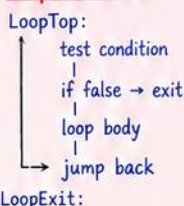
LoopTop:

```
sll $t1, $s3, 2 # t1 = i * 4
add $t1, $t1, $s6 # address of save[i]
lw $t0, 0($t1) # t0 = save[i]
bne $t0, $s5, LoopExit # exit if save[i] != k
addi $s3, $s3, 1 # i += 1
j LoopTop
LoopExit:
```

Array address:

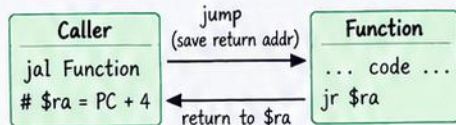
addr = base + 4 × i
(use sll to multiply index by 4)

Loop Structure



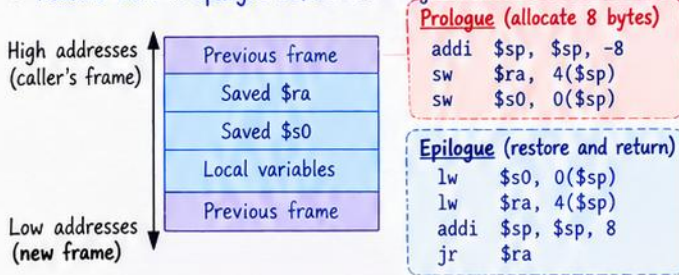
6. Procedure Calls (jal / jr \$ra)

- jal label → sets \$ra = PC + 4 and jumps.
- jr \$ra → returns to saved address.
- Arguments: \$a0 - \$a3
- Return value: \$v0 (and \$v1)
- Leaf function: no nested calls (can keep \$ra).
- Non-leaf function: **must** save \$ra before calling another function.



7. Stack Frames & Calling Convention

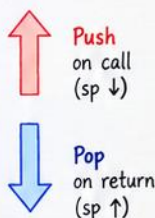
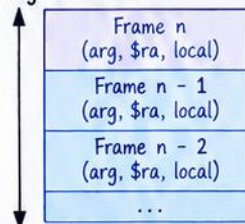
- Stack grows downward (high → low addresses).
- Non-leaf and recursive functions save \$ra and saved registers (e.g. \$s0) in prologue.
- Restore them in epilogue before returning.



8. Recursive Function Stack

- Each recursive call pushes its own frame with local argument and return address.
- Frames are popped in reverse order.

High addresses



9. MIPS Programming (SPIM) - Useful Instructions

Category	Instruction	Meaning
Load/Store	lw rd, mem	Load word (4 bytes)
	lb rd, mem	Load byte (sign-extended)
	sw rs, mem	Store word
	li rd, value	Load constant
Arithmetic	la rd, label	Load address of label
	add rd, rs, rt	rd = rs + rt
	sub rd, rs, rt	rd = rs - rt
	addi rd, rs, n	rd = rs + n
	mult rs, rt	64-bit product (Hi, Lo)
	div rs, rt	Lo = quotient, Hi = remainder
Branch/Jump	mfhi / mflo rd	Move from Hi / Lo
	beq / bne	branch if equal / not equal
	j label	unconditional jump
	jal label	\$ra = return addr, jump
	jr \$ra	return to caller

10. System Calls (SPIM)

Service	\$v0	Arguments	Description
Print integer	1	\$a0 = value	print int
Print string	4	\$a0 = addr	print string
Read integer	5	-	\$v0 = value
Read string	8	\$a0 = addr \$a1 = max+1	read string
Exit	10	-	terminate

Example: print integer in \$t2

```
li $v0, 1
move $a0, $t2
syscall
```

Example: print string

```
li $v0, 4
la $a0, msg
syscall
.data
msg: .asciiz "Hello\n"
```